

Geofence Crossing Workflow

Technical Guide

Boundary Crossing Detection — Methodology & Calculation Reference

Version 0.1.0 · Generated June 29, 2026

1. Overview

The **Geofence Crossing Workflow** detects and visualises the moments when tracked subjects cross the boundary of a user-defined region of interest (ROI). It ingests subject telemetry from **EarthRanger**, converts it to trajectories, intersects those trajectories with the ROI boundary, and delivers an interactive map and a per-subject table showing each crossing event with its time, direction, and coordinates.

The workflow is designed for any species or asset tracked via EarthRanger. The region of interest can be loaded from a local file, a remote URL, or directly from EarthRanger spatial features.

Note: Subject names are used throughout — the grouper defaults to `extra__subject__name`, which is populated automatically from EarthRanger subject details on every observation fetch.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All telemetry is fetched from an **EarthRanger** instance via `set_er_connection`. Subject group observations are retrieved with `get_subjectgroup_observations` over the configured time range. `raise_on_empty: false` allows the workflow to continue gracefully when no observations exist for the selected period.

2.2 Groupers

One grouper field is configured by default via `set_groupers`:

Grouper field	EarthRanger source	Typical use
<code>extra__subject__name</code>	<code>subject.name</code>	One table section per individual subject (default)

Additional groupers (sex, subtype, or any other subject attribute) can be added in the workflow configuration form. The grouper drives the `split_groups` step that partitions crossings into per-subject table views.

2.3 Region of Interest

`load_spatial_features_group` accepts three source types, selectable at runtime:

Source type	Description
<code>LocalFileSpatialFeatures</code>	GeoJSON, GeoPackage, or Shapefile from a local path
<code>RemoteFileSpatialFeatures</code>	GeoJSON or GeoPackage downloaded from a URL
<code>EarthRangerSpatialFeatures</code>	Spatial features fetched directly from the connected EarthRanger instance

The ROI must contain **Polygon** geometries defining the geofence boundary. All geometries are used; multi-polygon ROIs are supported.

2.4 Base Map Tile Layers

Base maps are configured via `set_base_maps` and composited beneath the data layers in the output ecomap.

3. Data Ingestion Pipeline

3.1 Observations → Trajectories

relocations_to_trajectory connects consecutive GPS fixes per subject into LineString segments. All extra__-prefixed subject detail columns (including extra__subject__name) are preserved on the trajectory GeoDataFrame and carried through to the crossing output.

Key columns on the resulting trajectory GeoDataFrame:

Column	Description
geometry	LineString connecting two consecutive fixes
groupby_col	Subject identifier used for grouping
segment_start	UTC datetime of the first fix in the segment
segment_end	UTC datetime of the second fix in the segment
extra__subject__name	Subject display name (from EarthRanger subject details)

3.2 Timezone Handling

get_timezone_from_time_range extracts the IANA timezone from the configured time range. After crossing detection, convert_values_to_timezone converts the crossing_time column from UTC to local time. The time range display format is %d %b %Y %H:%M:%S %Z.

4. Geofence Crossing Detection

4.1 Algorithm

detect_geofence_crossings iterates over every trajectory segment and tests for intersection with the ROI boundary. For each intersecting segment:

- The **crossing point geometry** is computed as the intersection of the segment LineString with the ROI boundary.
- The **crossing time** is interpolated linearly along the segment between segment_start and segment_end, proportional to the distance from the segment start to the crossing point.
- The **direction** is determined by whether the segment endpoint lies inside or outside the ROI polygon.
- All extra__-prefixed columns (e.g. extra__subject__name) are carried through to the crossing record.

4.2 Direction Classification

Direction value	Meaning	Map colour
Inward	Subject enters the ROI	#008000 (green)
Outward	Subject exits the ROI	#CC0000 (red)

apply_color_map maps the direction column to the two-colour palette above, storing the result in direction_colormap. This column drives the point layer fill colour on the map.

4.3 Crossing Output Columns

Column	Type	Description
geometry	Point	WGS 84 coordinate of the boundary crossing

Column	Type	Description
crossing_time	datetime	Interpolated local time of the crossing (after timezone conversion)
direction	str	"Inward" or "Outward"
longitude	float	Crossing point longitude, rounded to 6 decimal places
latitude	float	Crossing point latitude, rounded to 6 decimal places
direction_colormap	str	Hex colour string derived from direction
extra__subject__name	str	Subject display name, carried from the trajectory
groupby_col	str	Subject identifier, carried from the trajectory

5. Map Output

A single combined ecomap is produced showing the ROI boundary and all crossing events across all subjects. The map is interactive (pan, zoom, click tooltips) and saved as an HTML file.

5.1 ROI Boundary Layer

Property	Value
Geometry	Polygon outline (not filled)
Stroke colour	RGB(30, 30, 200, 200) — blue at 78 % opacity
Line width	2 px
Legend title	Region of Interest
Legend label	Boundary
Auto-zoom	Yes — map view fits to the ROI extent on load

5.2 Crossing Points Layer

Property	Value
Fill colour	From direction_colormap column (green = Inward, red = Outward)
Point radius	8 px
Legend	Derived from Direction column and direction_colormap
Tooltip columns	Crossing Time, Direction, Subject

5.3 Ecomap Settings

Property	Value
North arrow	Top-left placement
Legend	Title: "Crossings", bottom-right placement
Interactive	Yes (static: false)

Property	Value
Max zoom	20
Layer order	Base tiles → ROI boundary → crossing points

The map HTML is persisted to `$ECOSCOPE_WORKFLOWS_RESULTS` and referenced by the map widget in the dashboard.

6. Table Output

Per-subject crossing tables are produced by splitting the full crossings GeoDataFrame on the configured groupers, renaming columns for readability, then rendering each partition as an HTML table.

6.1 Column Renaming

Original column	Display name
crossing_time	Crossing Time
direction	Direction
extra__subject__name	Subject
longitude	Longitude
latitude	Latitude

Columns are renamed via `map_columns` with `raise_if_not_found: false` — missing columns are silently skipped rather than raising an error.

6.2 Table Widget

Each per-subject partition is rendered by `draw_table` with column order: **Crossing Time, Direction, Subject, Longitude, Latitude**. Each rendered HTML table is persisted, then wrapped in a `create_table_widget_single_view` titled *Geofence Crossings Table*. All per-subject views are merged into a single grouped table widget via `merge_widget_views`.

Note: Table widget tasks use `skipif: [never]` to ensure the dashboard always assembles, even when some subjects have no crossings.

7. Interactive Dashboard

`gather_dashboard` assembles the final dashboard from two widget groups, bound to the configured groupers and time range:

Widget	Type	Source
Geofence Crossings Map	Map	<code>ecomap_html_url</code> → <code>map_widget</code> → <code>merged_map_widget</code>
Geofence Crossings Table	Table	<code>crossings_table_html_url</code> → <code>table_widget</code> → <code>merged_table_widget</code>

The map widget presents all crossing events together on one combined view. The table widget presents per-subject views grouped by the active grouper, allowing users to navigate between subjects using the widget controls.

8. Output Files

All files are written to `$ECOSCOPE_WORKFLOWS_RESULTS`.

File / pattern	Format	Content
<code>crossings_map.html</code>	HTML	Interactive combined map — ROI boundary and all crossing points
<code>_crossings.html</code>	HTML	Per-subject crossing table (one file per subject group)

9. Workflow Execution Logic

9.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any_is_empty_df** — skips the task (and all dependants) when any input DataFrame is empty. This handles subjects with no observations, or trajectories that never cross the ROI boundary.
- **any_dependency_skipped** — propagates skips downstream automatically.

The `detect_geofence_crossings` task has an explicit additional skip condition to short-circuit cleanly when either the trajectory or the ROI is empty. Map and table widget tasks override defaults with `skipif: [never]` to ensure the dashboard always assembles.

9.2 Data Flow Summary

Stage	Tasks
Setup	EarthRanger connection, time range, timezone extraction, groupers, base maps
ROI loading	<code>load_spatial_features_group</code>
Telemetry ingest	<code>get_subjectgroup_observations</code> → <code>relocations_to_trajectory</code>
Crossing detection	<code>detect_geofence_crossings</code> → <code>convert_values_to_timezone</code> → <code>apply_color_map</code>
Grouping	<code>add_temporal_index</code> → <code>split_groups</code>
Map branch	<code>map_columns</code> (rename) → <code>create_polygon_layer</code> + <code>create_point_layer</code> → <code>draw_ecomap</code> → <code>persist</code> → widget
Table branch	<code>map_columns</code> (rename, per group) → <code>draw_table</code> → <code>persist</code> → widget (per group)
Dashboard	<code>gather_dashboard</code> combines map widget and table widget

10. Software Versions

Package	Version	Role
<code>ecoscope-workflows-core</code>	0.22.18.*	Core task library and workflow engine
<code>ecoscope-workflows-ext-ecoscope</code>	0.22.18.*	Spatial analysis tasks (relocations, trajectories, maps, widgets)

Package	Version	Role
ecoscope-workflows-ext-custom	>=0.0.46,<0.1.0	Custom tasks: geofence crossing detection, spatial feature loading

The core and ecoscope-ext packages are distributed via the `prefix.dev/ecoscope-workflows` conda channel. The custom package is built and distributed locally via a rattler-build artifacts channel. The runtime environment is managed by **pixi**.